

Rank 5 Vulnerability Fix — AI Session Log

AES-128 Peripheral: Soft-Reset Race Condition

Waylon Woo with Claude Opus 4.6 (1M context)

2026-04-12

Session Log: Rank 5 Vulnerability Fix

Environment: Claude Code (Opus 4.6, 1M context, high-effort mode) **Working directory:** ~/repo/projects/AI-HDL/main **Outcome:** Fix merged as commit 7e6276b on branch DP3.

Turn 1 — Read the security report, focus on Rank 5

User: Read Rank 5 vulnerability in DP3_submission/security_report.pdf and explain how we can fix it.

(Claude located DP3_submission/security_report.pdf, extracted all 13 pages, and found the Rank 5 section.)

Claude:

Rank 5 — HIGH: Soft Reset Race Condition

Problem (AES128/AES_memory.v:77-108): any software write to the control register with bit1 set triggers a full zeroisation — *even mid-encryption*.

Two concrete failure modes:

1. **Denial of service.** Any process with bus access can wipe the key at any time. On a multi-tenant system, one misbehaving task can destroy another’s key with a single 32-bit write.
2. **Encryption with a partially-live key.** If the reset fires while the key engine is mid-expansion, the key registers get zeroed but some round keys derived from the *original* key may already be latched in the pipeline. Subsequent rounds use zero-derived round keys. The resulting ciphertext mixes original-key- and zero-key-derived material — a cryptographically broken state that leaks information about the original key.

Proposed fix. Gate the soft reset on an “engine idle” interlock. The start bit (control_reg[0]) already tracks the busy/idle state — it is set when encryption begins and auto-cleared by done. Use it to suppress mid-encryption resets:

```
6'h00: begin
    // Ignore soft-reset bit while engine is busy (control_reg[0] == 1).
    if (data_in[1] && !control_reg[0]) begin
        reset_s <= 1'b1;
        control_reg <= 32'b0;
        key_mem[0] <= 32'b0;
        // ... rest of zeroisation ...
    end else begin
```

```

    // Normal control-reg write; mask out the reset bit so a busy-time
    // reset request is silently dropped rather than latched.
    control_reg <= data_in & ~32'h2;
    if (data_in[0]) active_bank <= ~active_bank;
end
end

```

Effort estimate: ~5 lines of RTL. **Why it works:** the engine will only accept a reset when idle, so no half-expanded round keys can ever reach the output.

Turn 2 — Apply the fix

User: Make those changes.

(Claude edited *AES128/AES_memory.v:79-94*, replacing the unconditional reset clause with the interlock above.)

Claude: Applied the interlock at *AES_memory.v:79-94*: soft reset is gated on `!control_reg[0]` (engine idle), and `bit1` is masked from control-register writes so busy-time reset requests are dropped rather than latched.

Turn 3 — Add a testbench to verify the interlock

User: Add a test to tb to test this works.

(Claude opened *AES128/testbenches/AES_memory_tb.v*, studied the existing helper tasks (*write32*, *check32*, *check_sig*, *check_key*, *engine_done*, *do_reset*), and appended a new *TEST 5*.)

TEST 5 outline:

1. Load a known key and plaintext.
2. Assert `start`, putting the engine in the busy state.
3. Write `0x02` (reset only). Expect `reset_s` low, `start` high, key intact.
4. Write `0x0E` (reset + `key_lock` + `auto_start`). Expect `reset_s` still low, key still intact — reset bit masked, other bits may land.
5. Signal `done`, returning the engine to idle.
6. Write `0x02` again. Expect `reset_s` pulse and the key zeroed.

Compile and run:

```

iverilog -g2012 -o outputs/test_sim \
    AES128/testbenches/AES_memory_tb.v AES128/AES_memory.v
vvp outputs/test_sim

```

First-run result:

```

=== TEST 5: Soft reset interlock while engine busy ===
PASS t5_start_asserted => 1
PASS t5_reset_suppressed_while_busy => 0
FAIL t5_start_still_asserted: got 0 expected 1
FAIL t5_reset_bit_masked_in_combined_write: got 1 expected 0
FAIL original_key: got 0x0...0 expected 0x9abc...beef

```

Claude: The test caught a subtlety. My mask (`data_in & ~0x2`) let the CPU clear the *start* bit as a side effect: writing `0x02` also writes 0 to `bit0`, which de-asserts `control_reg[0]`. A subsequent write of `0x0E` then sees the engine as *idle* and the reset fires — wiping the key.

Two options:

- (a) Scope the test to the strict Rank 5 invariant and accept that a naive CPU can still kill `start` via control-reg writes.
- (b) Tighten the fix so the start bit is *pinned* during busy; CPU writes cannot disturb an in-flight encryption.

Option (b) is more robust — the Rank 5 failure mode is really about “mid-encryption interference by the CPU,” and pinning `start` closes the whole class, not just the reset variant. Going with (b).

Turn 3.5 — Tighten the fix

(Claude replaced the single-mask with a two-branch structure that preserves `control_reg[0]` during busy-time writes.)

```
6'h00: begin
    // Rank 5 interlock: while the engine is busy
    // (control_reg[0] == 1), suppress CPU control-reg writes
    // that would disturb an in-flight encryption.
    // - bit1 (soft reset): dropped - prevents DoS wipes and
    //   mid-expansion key corruption (leaks original key).
    // - bit0 (start): preserved - a CPU write of 0 must not
    //   silently de-assert start mid-run; start is cleared
    //   only by `done`.
    // Other bits (key_lock, auto_start) are safe to update.
    if (control_reg[0]) begin
        control_reg <= {data_in[31:2], 1'b0, control_reg[0]};
    end else begin
        control_reg <= data_in;
        if (data_in[0]) begin
            active_bank <= ~active_bank;
        end
    end
    // Soft reset is only honored when the engine is idle.
    if (data_in[1] && !control_reg[0]) begin
        reset_s <= 1'b1;
        // ... zeroise key_mem, plaintext banks, ciphertext, control_reg ...
    end
end
```

Second-run result:

```
=== TEST 5: Soft reset interlock while engine busy ===
PASS original_key => 0x9abcdef012345678cafebabedeadbeef
PASS t5_start_asserted => 1
PASS t5_reset_suppressed_while_busy => 0
PASS t5_start_still_asserted => 1
PASS original_key => 0x9abcdef012345678cafebabedeadbeef
PASS t5_reset_bit_masked_in_combined_write => 0
PASS original_key => 0x9abcdef012345678cafebabedeadbeef
PASS t5_idle_after_done => 0
PASS t5_reset_honored_when_idle => 1
PASS original_key => 0x00000000000000000000000000000000
```

=====

PASSED: 60
FAILED: 0
ALL TESTS PASSED

=====

All 60 assertions (51 pre-existing + 9 new Rank 5 checks) pass. Tests 1–4 still pass, confirming no regression to the base write/read, auto-start, double-buffering, or key-reuse flows.

Turn 4 — Commit

User: add and commit these changes

(Claude ran `git status`, staged the two modified files, and created a new commit. No `--amend`, no `force-push`.)

```
git add AES128/AES_memory.v AES128/testbenches/AES_memory_tb.v
git commit -m "Fix Rank 5 vulnerability: soft-reset race condition ..."
```

Result: [DP3 7e6276b] Fix Rank 5 vulnerability: soft-reset race condition (2 files changed, 68 insertions, 7 deletions).

Before / After

Property	Before	After
Reset during idle	Zeroises state	Zeroises state
Reset during busy	Zeroises	Dropped silently
CPU can clear <code>start</code> mid-encryption	Yes	No (pinned)
DoS via bus reset	Yes	Mitigated (busy window)
Key leakage via mid-expansion reset	Yes	Mitigated

Summary of artifacts

Files changed

File	Change
AES128/AES_memory.v	Control-register write path gated on engine-idle; start bit pinned during busy
AES128/testbenches/AES_memory_tb.v	Added TEST 5 (9 new asserts) covering the full interlock behaviour

Verification

- `run_tb.py` option 5 (memory TB): **60/60 asserts pass** (51 existing + 9 new Rank 5 interlock assertions).
- No regression in Tests 1–4 (basic read/write, auto-start, double-buffering, key reuse).

Commit

Commit	Subject
7e6276b	Fix Rank 5 vulnerability: soft-reset race condition

Branch: DP3.

Process note

The first pass of the fix was *functionally correct for the stated Rank 5 threat* (reset is suppressed while busy) but left a side-channel for CPU interference with `start`. Writing the test before declaring the fix complete surfaced that gap immediately — a reminder that “the named attack is closed” and “the surrounding class is closed” are different claims, and a good test discriminates between them.

Out of scope

Ranks 3, 4, 6, 7, 8 from the security report remain unaddressed in this session. See `DP3_submission/security_report.pdf` for details.